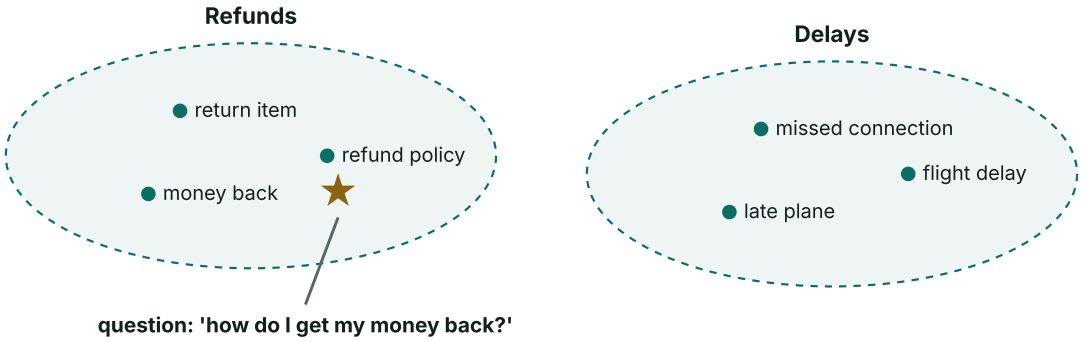
Embeddings, Retrieval & Vector Search

1. What an embedding is

An embedding is a fixed-length **list of numbers that captures meaning**. It's learned with **contrastive training**: the model sees pairs that belong together (a question and its answer) and pairs that don't, until belonging together means being close together. Once meaning is geometry, search becomes arithmetic.

- Similar meaning → vectors land **close together**
- Embeddings are **one-way**: you can't turn a vector back into text, which is why the original text is stored next to it

✎ DIAGRAM



Similar meanings sit close together, so the question lands in the refund cluster even with different words.

2. The retrieval flow

1. Chunk the documents
2. Embed each chunk
3. Store vector + **original text** + metadata in a vector database
4. Embed the user's question
5. Find the nearest vectors (cosine similarity)
6. Pass their **text** to the LLM

- **Cosine similarity** is the near-universal text default; always use the metric the model's documentation specifies (a mismatch silently degrades retrieval)
- For Arabic or mixed corpora use an explicitly **multilingual** model, which also gives cross-lingual search (an English question finds an Arabic document)

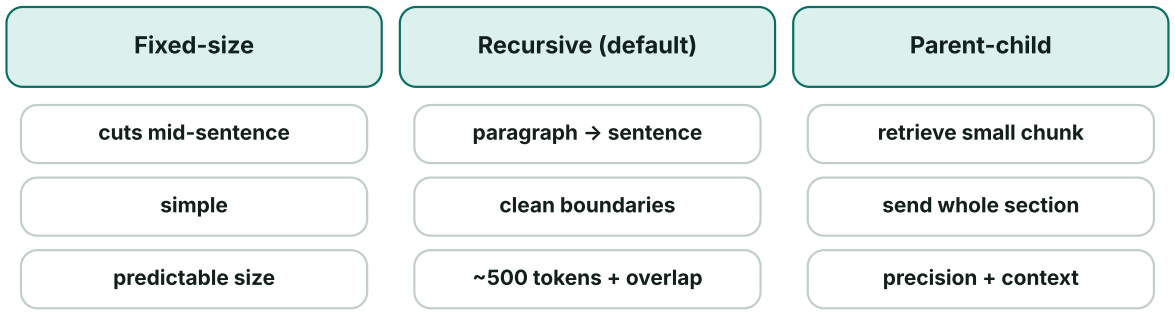
3. Chunking: the decision that sets the ceiling

The chunk is the unit you retrieve. Too big blurs several topics into one vector; too small loses context (a chunk saying '14 days' with no product name). **Most bad RAG is bad chunking.**

1. **Fixed-size**: every N tokens + overlap. Simple, cuts sentences in half
2. **Recursive** (default): split on paragraphs → sentences → words until under the limit
3. **Sentence / paragraph**: natural boundaries, uneven sizes
4. **Structure-aware**: split on headings, sections, slides, functions. Best for manuals, policies, code
5. **Semantic**: embed sentences, cut where meaning shifts. Costlier; test it
6. **Parent-child (small-to-big)**: retrieve small precise chunks, send the whole parent section to the LLM
7. **Sentence window**: retrieve one sentence plus its neighbours
8. **Contextual chunking**: prepend an LLM-written line saying where the chunk sits ('From the 2026 refund policy, electronics section')
9. **Proposition**: an LLM rewrites text into small standalone facts. Precise, expensive
10. **Late chunking**: embed the whole document first, then split, so each chunk keeps document context

- Knobs: size ~200 to 1,000 tokens (default 400 to 600), **10 to 20% overlap**, within the embedding model's input limit
- Metadata on every chunk: source, title, **heading path**, date, language, permissions
- Awkward content: keep **tables** whole or one row per chunk with headers repeated; fix **PDF parsing** first; **FAQs** as one question + answer per chunk; **code** by function

✎ DIAGRAM



Start recursive; switch to structure-aware or parent-child if the golden set shows chunks lacking context.

✎ DIAGRAM

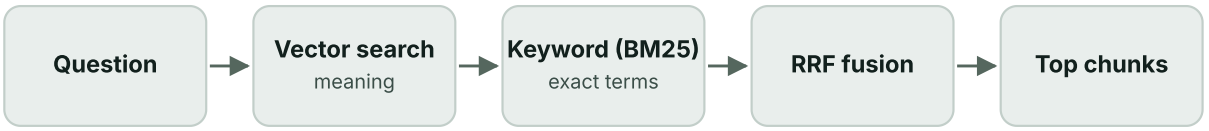
Symptom	Likely fix
Answer split across two chunks	More overlap, or parent-child
Chunk makes no sense alone	Heading breadcrumbs or contextual chunking
Tables come back garbled	Structure-aware parsing
Right document, wrong section	Smaller chunks plus reranking

Chunking problems show up as recognisable symptoms. Compare configs on recall@K.

4. Hybrid search

- Embeddings match meaning but **miss exact strings**: product codes, error codes, names, acronyms
- **BM25** (keyword search) catches exactly those
- Run both and merge with **reciprocal rank fusion (RRF)** → typically a 10 to 20% recall lift at trivial cost. Pure vector search is an anti-pattern at scale

✎ DIAGRAM



Hybrid search runs both searches and merges the ranked lists.

5. Vector databases and indexes

- Options: **pgvector** (default if Postgres is already there), **Qdrant** (open source, strong filtering), **Pinecone** (zero-ops). Pick by operational fit
- **Flat**: exact, checks everything (small sets) · **IVF**: searches the nearest clusters · **HNSW**: layered graph, fast, 95 to 99% recall, memory-hungry (the default) · **PQ**: compresses vectors
- **Metadata filters**: pre-filter (accurate) vs post-filter (can leave too few results) · **MMR** diversifies results so you don't get five near-duplicates
- **Bi-encoder** (separate embeddings, fast retrieval) vs **cross-encoder** (reads both together, accurate reranking) vs **ColBERT** (word-level, a middle ground)

6. Vector maths

- Dimensions: 384 · 768 · 1,024 · 1,536 · 3,072. More = more nuance, but more storage and slower search. **Matryoshka** embeddings can be shortened with little loss
- Storage = vectors × dimensions × **4 bytes** (float32)
- **Quantisation** shrinks it: float16 = ½, int8 = ¼, binary = 1/32, with some recall loss

✎ DIAGRAM

```
1,000,000 vectors × 1,536 dims × 4 bytes
= 6,144,000,000 bytes
≈ 6 GB (+ index overhead)
```

The storage sum to be able to do on a whiteboard.

7. Retrieval metrics

- **recall@K**: was the right chunk anywhere in the top K? (check this first)
- **precision@K**: how much of the top K was relevant?
- **MRR**: how high was the first correct result? · **nDCG**: were the best results ranked highest?

SAY IT

"An embedding is a list of numbers that captures meaning, so similar texts land close together. I'd chunk recursively at about 500 tokens with heading metadata, run hybrid search, and judge it on recall@K."